PulsePass — Product Requirements Document

Plataforma de eventos, artistas y entradas

Caso de estudio académico — Fase 4: Capa de Seguridad con Spring Security y JJWT

Campo	Detalle
Documento	PRD — PulsePass — Capa de Seguridad
Versión	1.0
Estado	Propuesto para implementación académica
Fecha	8 de octubre de 2026
Tecnologías objetivo	Java 21, Spring Boot 4.x, Spring Security 7.x, JJWT 0.13.x, Maven, PostgreSQL, Flyway, JUnit 5, Mockito, MockMvc, Testcontainers
Alcance técnico	Autenticación, autorización, JWT firmado, identidad de usuario, seguridad de las 20 operaciones REST y pruebas
Predecesores	Fase 1 Persistencia → Fase 2 Servicios → Fase 3 Controladores REST

Control del documento

Campo	Definición
Propósito	Definir requisitos funcionales, decisiones de seguridad, contratos HTTP, criterios de aceptación y pruebas para incorporar seguridad al sistema PulsePass ya construido.
Audiencia	Estudiantes universitarios, docente, equipo de desarrollo y equipo QA.
Dependencias	PRD de Persistencia; PRD de Servicios; PRD de Controladores REST .
Estrategia	Extensión incremental: preservar las reglas de negocio, 20 endpoints y DTOs de dominio; modificar solo lo imprescindible para gestionar credenciales y el contexto del usuario autenticado.
Decisiones de MVP	JWT de acceso firmado con HS256; tres roles (`USER`, `STAFF`, `ADMIN`);

API stateless mediante encabezado Bearer.

---

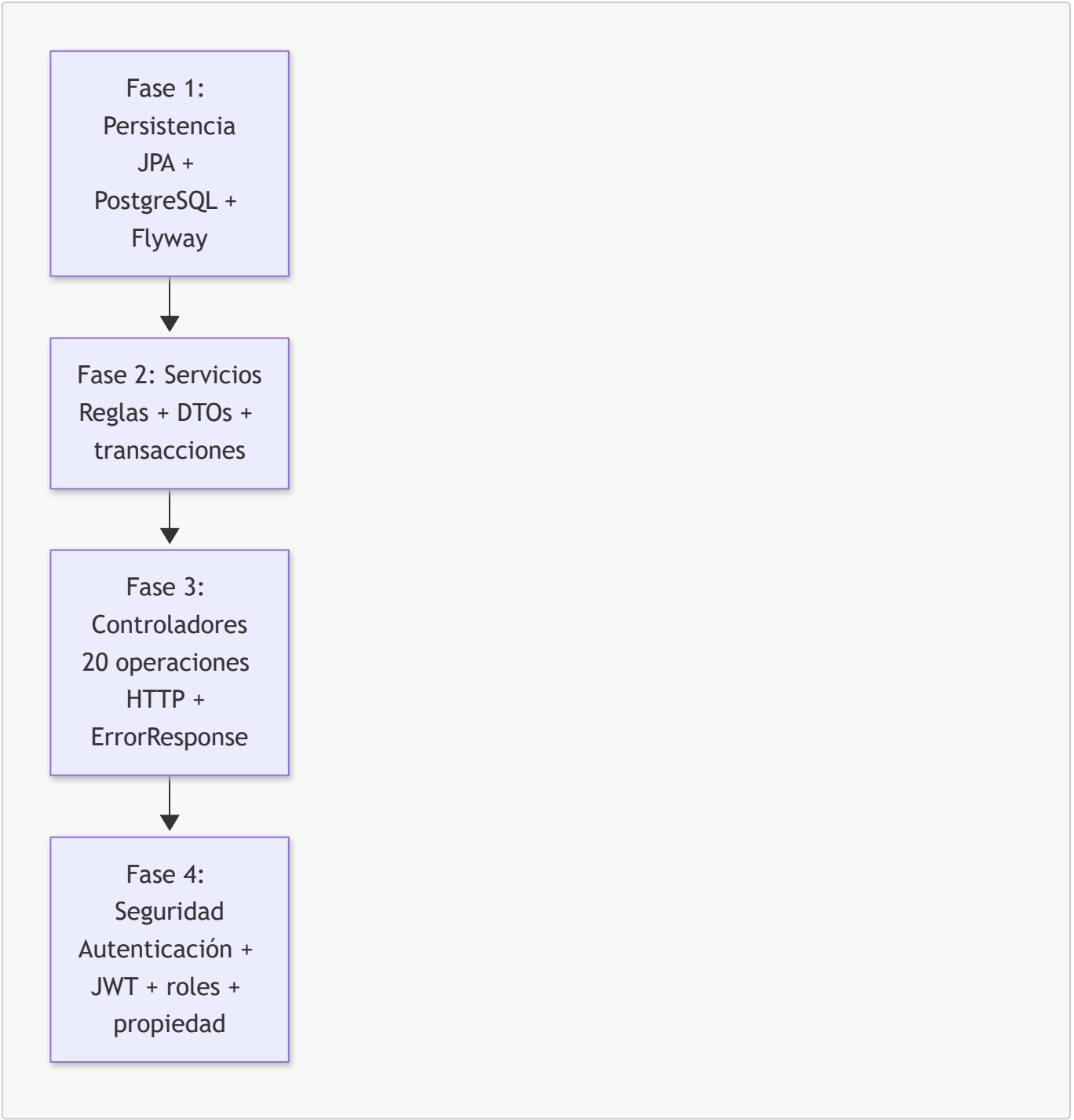
1. Resumen ejecutivo

PulsePass permite descubrir eventos y gestionar entradas a conciertos, festivales, conferencias, competencias deportivas y actividades culturales. Las tres fases previas implementaron persistencia relacional, reglas de negocio en Services y una API REST con 20 operaciones. La API carece todavía de un mecanismo para comprobar **quién realiza una solicitud** y **qué operaciones puede ejecutar**.

Esta cuarta fase incorpora autenticación basada en credenciales, emisión y validación de JWT utilizando **JJWT**, autorización basada en roles con **Spring Security**, verificación de la propiedad de recursos y tratamiento uniforme de errores. Debe impedirse que un usuario opere sobre tickets ajenos aunque conozca su código o el correo de otro usuario.

> **Principio de producto:** una solicitud autenticada no implica acceso ilimitado. Cada operación debe cumplir una política de rol y, cuando corresponda, una política de propiedad del recurso.

1.1 Evolución por fases



La Fase 4 **no reemplaza** las fases anteriores. Añade un perímetro HTTP seguro y mínimos cambios compatibles para representar contraseñas, roles y usuarios activos.

2. Visión y problema

2.1 Problema

Sin autenticación y autorización, cualquier cliente podría intentar registrar eventos, publicar eventos, acceder a información personal, consultar entradas de otros usuarios, cancelarlas o marcarlas como utilizadas. Proteger un endpoint únicamente porque contiene una URL difícil de adivinar no es suficiente.

2.2 Visión

Construir una API REST *stateless*, verificable y fácil de probar que reconozca al usuario autenticado, aplique permisos explícitos y mantenga intactas las reglas funcionales implementadas en los Services.

2.3 Objetivos medibles

1. Permitir registro e inicio de sesión con contraseña almacenada como hash seguro.
2. Emitir JWT firmados y de corta duración, y verificar su integridad y vigencia.
3. Exigir Bearer token para todas las rutas que no sean expresamente públicas.
4. Controlar los 20 endpoints existentes por rol y por propiedad de los datos.
5. Retornar `401` y `403` correctamente sin romper `ErrorResponse`.
6. Mantener aisladas las responsabilidades de Controller, Service y Repository.
7. Automatizar pruebas de los caminos positivos, negativos y de seguridad.

2.4 No objetivos

No se implementarán OAuth2 Authorization Server, SSO, Keycloak, redes sociales, MFA, recuperación de contraseña por correo, refresh tokens, cookies de sesión, bloqueo distribuido, revocación individual instantánea, microservicios, pasarela de pagos ni frontend. Esas capacidades pueden añadirse en fases posteriores; no deben presentarse como implementadas.

3. Relación con los PRD anteriores

3.1 Fase 1 — Persistencia

Modelo existente, **sin eliminación ni renombrado**:

```
Venue    Event    Artist    User    UserProfile    Ticket
```

Relaciones: `Venue 1:N Event`, `Event N:M Artist`, `User 1:1 UserProfile`, `User 1:N Ticket`, `Event 1:N Ticket`.

Enums existentes: `EventCategory`, `EventStatus`, `TicketType`, `TicketStatus`. Se mantienen PostgreSQL, Spring Data JPA, Flyway y validación del esquema por Hibernate.

Ampliación obligatoria: nueva migración versionada (posterior a las V1–V3 existentes) para atributos de autenticación y autorizaciones. No alterar ni reescribir migraciones ya ejecutadas.

3.2 Fase 2 — Servicios

Se preservan interfaces y contratos funcionales `VenueService`, `EventService`, `ArtistService`, `UserService` y `TicketService`, así como sus implementaciones, DTOs `record`, mapeadores MapStruct, transacciones y excepciones. Los Services siguen siendo responsables de las reglas: edad mínima, capacidad, evento publicado, usuario activo, duplicados y transiciones de tickets.

Los controles de **propiedad** deben situarse en una frontera de negocio segura (preferentemente un servicio de autorización o una anotación de seguridad a nivel de método), no depender solamente del Controller. El

servicio de autenticación puede usar un repositorio de usuarios y componentes de seguridad sin convertirse en un nuevo sustituto de `UserService`.

3.3 Fase 3 — Controladores REST

Se conservan las **20 operaciones originales** (2 Venue, 6 Event, 3 Artist, 3 User y 6 Ticket) y sus DTOs de respuesta. `GlobalExceptionHandler` sigue encargándose de las excepciones de negocio y Bean Validation. `AuthenticationEntryPoint` y `AccessDeniedHandler` transforman en JSON los errores originados en el filtro de seguridad.

La Fase 3 deliberadamente excluía Spring Security y JWT; la presente fase agrega ambas capacidades de manera explícita.

3.4 Compatibilidad y cambios inevitables

Elemento previo	Estado en Fase 4
20 rutas existentes	Se conservan URL, verbo y respuesta exitosa.
`RegisterUserRequest`	Se extiende con `password` obligatoria para un registro nuevo; *cambio de contrato de entrada documentado*.
`UserResponse`	Se mantiene, sin hash, contraseña ni secretos.
`PurchaseTicketRequest.userEmail`	Se mantiene por compatibilidad, pero se valida que coincida con la identidad autenticada para `USER`.
Búsquedas por `email`/`username`	Se conservan rutas, con restricciones de identidad y rol.
`ErrorResponse`	Se mantiene exactamente el contrato de seis campos indicado en la Fase 3.
Reglas existentes de Service	Se conservan; se añaden precondiciones de autorización cuando aplica.
Base de datos	Migración incremental; no destruir datos anteriores.

4. Actores y necesidades

Actor	Necesidad
Visitante anónimo	Ver información pública de venues, artistas y eventos publicados; registrarse e iniciar sesión.
`USER`	Comprar entradas, consultar sus propios datos y tickets, y cancelar sus propios tickets bajo las reglas previas.
`STAFF`	Operar eventos y controlar el ingreso mediante validación de entradas.
`ADMIN`	Administrar eventos, asociar artistas y consultar datos operativos sensibles.
Cliente HTTP	Enviar JWT en `Authorization: Bearer <token>` y recibir errores JSON coherentes.

Actor	Necesidad
QA	Demostrar restricciones por rol y propiedad mediante pruebas automatizadas.

5. Alcance del MVP

5.1 Incluido

- `spring-boot-starter-security`, `SecurityFilterChain` y `authorizeHttpRequests`.
- Autenticación por usuario o correo y contraseña mediante `AuthenticationManager` / `DaoAuthenticationProvider` o equivalente compatible.
- `UserDetailsService`, `PasswordEncoder` BCrypt y validación de estado activo.
- JWT firmados HS256 con JJWT y clave secreta de longitud suficiente, cargada desde el entorno.
- Filtro Bearer único por request (`OncePerRequestFilter`) y configuración sin sesión.
- Tres roles: `ROLE_USER`, `ROLE_STAFF`, `ROLE_ADMIN`.
- Verificación de propiedad de tickets y de datos personales.
- `AuthenticationEntryPoint`, `AccessDeniedHandler`, `ErrorResponse`, manejo de entradas malformadas.
- Migraciones Flyway para credenciales/roles y estrategia explícita para datos legados.
- Pruebas unitarias, MVC con Spring Security y de integración selectivas.

5.2 Fuera de alcance

- Contratos de pago reales; modificación de reglas comerciales o de eventos.
- MFA, rotación automática de claves, gestión completa de sesiones y logout con lista de revocación.
- Cambio de contraseña y recuperación automatizada.
- Integración con proveedor externo de identidad.
- Despliegue productivo completo, aunque se documentan controles necesarios para producción.

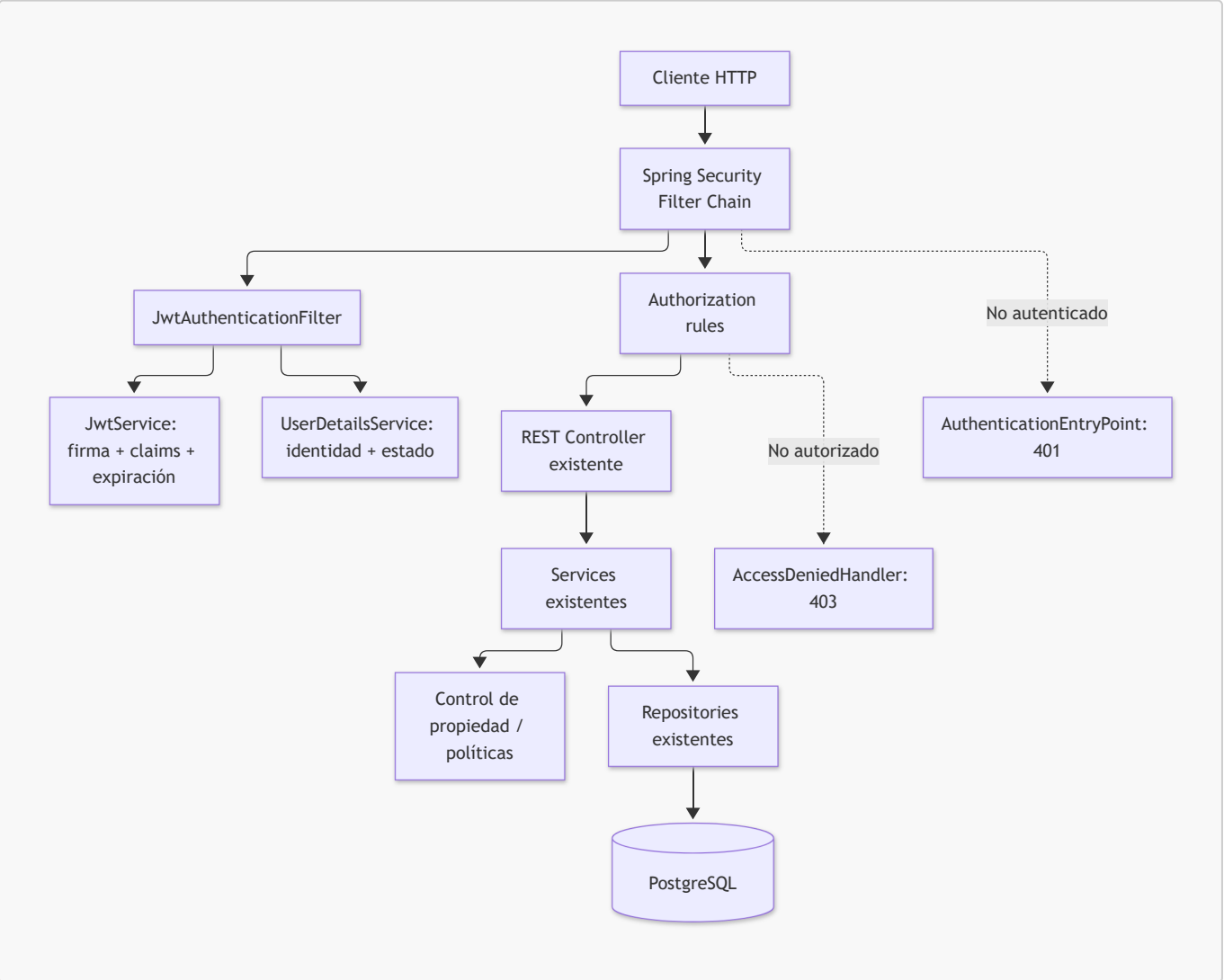
6. Fundamentos que el estudiante debe comprender

Concepto	Definición y consecuencia práctica
Autenticación	Confirmar identidad a partir de credenciales válidas.
Autorización	Decidir qué puede hacer un usuario ya autenticado.
Principal	Identidad comprobada de quien realiza la solicitud.
Authority	Permiso manejado por Spring Security; `ROLE_USER` es una authority.
Role	Agrupación de permisos; `hasRole("ADMIN")` comprueba `ROLE_ADMIN`.
Hash de contraseña	Resultado irreversible diseñado para verificar contraseñas; no es cifrado reversible.
JWT	Token compacto con cabecera, claims y firma; su contenido no está cifrado.
JWS	JWT firmado que permite detectar manipulación.
Stateless	No se almacena sesión de autenticación del usuario en el servidor.

Concepto	Definición y consecuencia práctica
401 / 403	401: falta identidad válida; 403: identidad válida sin permiso.
Propiedad	Regla que relaciona el recurso solicitado con la identidad autenticada.

> La firma no oculta los claims del JWT. Nunca incluir contraseña, documento personal, correo privado innecesario ni datos sensibles en el payload.

7. Arquitectura esperada



7.1 Separación de responsabilidades

Componente	Responsabilidad	Prohibido
`SecurityConfig`	Declarar filtro, autenticación y reglas de rutas.	Negocio de compra/publicación.
`JwtService`	Emitir y validar firma, issuer, tipo, fechas y sujeto.	Consultar tickets o ejecutar negocio.

Componente	Responsabilidad	Prohibido
`JwtAuthenticationFilter`	Leer Bearer, validar JWT, construir contexto seguro.	Crear cuentas o persistir tokens de acceso.
`CustomUserDetailsService`	Cargar identidad, password hash, roles y estado de usuario.	Validar reglas comerciales.
`AuthService`	Autenticar, registrar credenciales según diseño, emitir token.	Reemplazar todos los Services anteriores.
`AuthorizationService`	Comparar principal con propietario del recurso y comprobar permisos.	Asumir que un ID enviado por cliente es confiable.
Controllers	Recibir requests y delegar.	Interpretar JWT manualmente.
Services	Negocio, transacciones y validaciones de propiedad cuando corresponda.	Usar contraseñas en texto plano.

8. Principios de diseño obligatorios

ID	Principio
SEC-001	Denegar por defecto: `anyRequest().authenticated()` o `denyAll()` según definición explícita; ninguna ruta accidentalmente pública.
SEC-002	Exponer públicamente solo rutas aprobadas.
SEC-003	Contraseñas BCrypt, nunca en texto plano ni logs.
SEC-004	Clave JJWT desde entorno; prohibida en código fuente.
SEC-005	JWT firmado, con expiración y validación de emisor.
SEC-006	No confiar en `role`, `userEmail` o `userId` proporcionados por el cliente como identidad.
SEC-007	`ROLE_USER` puede operar únicamente sobre sus recursos.
SEC-008	JWT inválido o vencido no concede acceso.
SEC-009	Respuestas 401/403 uniformes y sin filtración de detalles internos.
SEC-010	Mantener separadas autenticación, autorización y reglas comerciales.

ID	Principio
SEC-011	Un usuario desactivado pierde acceso en el siguiente request verificado contra su estado.
SEC-012	Las pruebas cubren rutas, roles, propiedad y regresión funcional.

9. Modelo de identidad y migración

9.1 Diseño de datos

Se conserva `User` como entidad de negocio. La identidad de login se vincula con su ID persistente, **no** con un correo editable como identificador técnico. Diseño mínimo recomendado:

```
User (existente)
 ├── id
 ├── username (único)
 ├── email (único)
 ├── profile (existente)
 └── estado/active (reutilizar si ya existe)

UserCredential (NUEVA, 1:1 con User)
 ├── user_id (PK y FK unique)
 ├── password_hash (NOT NULL)
 └── password_changed_at (opcional)

UserRole (NUEVA, N:1 con User)
 ├── user_id (FK)
 └── role (USER | STAFF | ADMIN)
  PK(user_id, role)
```

Esta separación evita modificar indebidamente DTOs de negocio y elimina cualquier posibilidad de serializar `passwordHash` desde MapStruct por accidente. Si el PRD de persistencia ya incluye campos equivalentes, reutilizarlos.

9.2 Migración Flyway

- Crear `V4__security_credentials_roles.sql` o la **siguiente versión libre** posterior a las ya existentes; nunca reutilizar una versión aplicada.
- Crear tabla `user_credentials` con relación 1:1, `password_hash` y constraints.
- Crear tabla `user_roles` con rol restringido y PK compuesta.
- Añadir índices cuando el plan de consulta lo justifique; conservar `username` y `email` únicos.
- Crear procedimiento seguro para vincular credenciales de usuarios preexistentes **sin asignar contraseñas conocidas o compartidas**.
- Los registros legados sin credenciales quedarán **sin posibilidad de login** hasta una activación segura; no se debe inventar un password predeterminado.

- No promover a `ADMIN` durante el registro público. Los roles privilegiados se provisionan por proceso administrativo o semilla local controlada, no por parámetros HTTP del cliente.

9.3 Alta y registro

`POST /api/users` mantiene su URL y semántica de creación de cuenta. `RegisterUserRequest` incorpora `password` y se validará longitud razonable (mínimo académico: **12 caracteres**, máximo 72 bytes si se emplea BCrypt tradicional), sin devolverla en el `UserResponse`. `UserService.register()` sigue aplicando unicidad y validaciones de negocio. `AuthService` debe orquestar la creación de usuario y hash en una única transacción, evitando cuentas creadas sin credenciales. Si se conserva la invocación directa de `UserService.register()` desde Controller, se debe refactorizar de forma explícita para garantizar atomicidad.

10. Requisitos funcionales — autenticación

ID	Requisito	Prioridad	Criterio de aceptación
FR-SEC-AUTH-001	Registro público	Must	Crea `User` + credencial BCrypt + `ROLE_USER` de forma atómica; 201.
FR-SEC-AUTH-002	Login	Must	Autentica usuario/correo + contraseña; 200 con token y metadatos.
FR-SEC-AUTH-003	Contraseña incorrecta	Must	Retorna 401 genérico, no informa si el usuario existe.
FR-SEC-AUTH-004	Usuario inactivo	Must	No obtiene token ni accede con uno previamente emitido si se verifica estado en cada solicitud.
FR-SEC-AUTH-005	Emisión JWT	Must	Contiene `sub`, `iss`, `iat`, `exp`, `jti`; firma válida; expiración configurada.
FR-SEC-AUTH-006	Validación JWT	Must	Rechaza JWT expirado, manipulado, issuer erróneo, token malformado o algoritmo inesperado.
FR-SEC-AUTH-007	Bearer	Must	Token solo por encabezado `Authorization`; no aceptar JWT en query string.
FR-SEC-AUTH-008	Estado sin sesión	Must	La API no usa `JSESSIONID` como autenticación.
FR-SEC-AUTH-009	Contexto de seguridad	Must	El principal se construye usando usuario persistido y authorities confiables.
FR-SEC-AUTH-010	Usuario autenticado	Should	`GET /api/auth/me` retorna información mínima del principal, no secretos.

10.1 Nuevos endpoints de autenticación

Verbo	Ruta	Acceso	Request	Resultado
POST	`/api/auth/login`	Público	`LoginRequest(identifier, password)`	`200 AuthResponse(accessToken, tokenType, expiresIn)`
GET	`/api/auth/me`	Autenticado	Bearer	`200 AuthenticatedUserResponse`
POST	`/api/users`	Público	`RegisterUserRequest` extendido	`201 UserResponse` (contrato anterior)

`LoginRequest`, `AuthResponse` y `AuthenticatedUserResponse` son nuevos `record`. No devolver el hash ni la contraseña, ni emitir token automáticamente al registrar, salvo decisión posterior expresamente registrada. Para el MVP se adopta **registro → login**.

10.2 Ejemplos HTTP

```
POST /api/auth/login
Content-Type: application/json

{
  "identifier": "andrea",
  "password": "UnaFraseSeguraLarga2026!"
}
```

```
{
  "accessToken": "<JWT_FIRMADO>",
  "tokenType": "Bearer",
  "expiresIn": 900
}
```

```
GET /api/auth/me
Authorization: Bearer <JWT_FIRMADO>
```

La duración inicial de acceso se define en **900 segundos (15 minutos)**, parametrizable. Es un valor de diseño del MVP, no una regla heredada.

11. Requisitos funcionales — autorización de las 20 operaciones anteriores

Esta matriz es el **contrato de seguridad obligatorio** que se agrega a los endpoints ya existentes. `STAFF` y `ADMIN` se consideran identidades privilegiadas para acciones indicadas, **no** propietarios automáticos de todos

los recursos en las rutas de autoservicio. El acceso administrativo a recursos ajenos debe estar expresamente autorizado.

ID	Método y ruta heredada	Regla Fase 4
FR-SEC-VEN-01	`GET /api/venues/{code}`	Público.
FR-SEC-VEN-02	`GET /api/venues/active`	Público.
FR-SEC-EVT-01	`POST /api/events`	`ADMIN`.
FR-SEC-EVT-02	`GET /api/events/{eventCode}`	Público solo si publicado; para otros estados `STAFF`/`ADMIN`, evitando revelar borradores.
FR-SEC-EVT-03	`GET /api/events/published`	Público.
FR-SEC-EVT-04	`PATCH /api/events/{eventCode}/publish`	`ADMIN`.
FR-SEC-EVT-05	`POST /api/events/{eventCode}/artists/{artistId}`	`ADMIN`.
FR-SEC-EVT-06	`GET /api/events/by-artist?stageName=...`	Público únicamente para eventos publicados; otros estados requieren `STAFF`/`ADMIN`.
FR-SEC-ART-01	`GET /api/artists/{id}`	Público.
FR-SEC-ART-02	`GET /api/artists/by-stage-name?stageName=...`	Público.
FR-SEC-ART-03	`GET /api/artists/active`	Público.
FR-SEC-USR-01	`POST /api/users`	Público (rol inicial exclusivamente `USER`).
FR-SEC-USR-02	`GET /api/users/by-email?email=...`	Propio usuario, o `ADMIN` con propósito autorizado.
FR-SEC-USR-03	`GET /api/users/by-username?username=...`	Propio usuario, o `ADMIN` con propósito autorizado.
FR-SEC-TKT-01	`POST /api/tickets`	`USER`; `userEmail` debe pertenecer al principal.
FR-SEC-TKT-02	`GET /api/tickets/{ticketCode}`	Propietario, o `STAFF`/`ADMIN` en operación autorizada.

ID	Método y ruta heredada	Regla Fase 4
FR-SEC-TKT-03	`GET /api/tickets/by-user?email=...`	Propietario, o `ADMIN` autorizado.
FR-SEC-TKT-04	`GET /api/events/{eventCode}/tickets/paid`	`STAFF`/`ADMIN`.
FR-SEC-TKT-05	`PATCH /api/tickets/{ticketCode}/cancel`	Propietario (`USER`) o `ADMIN` expresamente autorizado; sigue vigente la regla de cancelación del Service.
FR-SEC-TKT-06	`PATCH /api/tickets/{ticketCode}/use`	`STAFF`/`ADMIN`; conserva transición válida del Service.

Nota de trazabilidad: las cinco tablas de controlador suman 20 operaciones. El segundo endpoint de Venue se conserva como `GET /api/venues/active`, correspondiente a `findActiveVenues()`. Si `findByCode(eventCode)` o `findByArtist(stageName)` hoy exponen eventos no publicados, su acceso público requiere filtrado explícito sin alterar la información accesible a `STAFF`/`ADMIN`.

11.1 Requisitos de propiedad (antí-IDOR)

- `GET /api/tickets/{ticketCode}` debe determinar el usuario propietario desde la base de datos y contrastarlo con `principal.userId`; jamás basta el `ticketCode`.
- `GET /api/tickets/by-user?email=...` y las dos búsquedas de usuario deben comparar el usuario solicitado con la identidad autenticada usando ID persistente, no solo un string de email.
- `POST /api/tickets` debe comprobar que el `userEmail` del DTO representa al principal (para `USER`). Nunca confiar en el email del request como comprobación de autorización.
- `cancel` realiza autorización de propiedad antes de mutar estado, preservando además todas las condiciones del servicio previo.
- `STAFF` puede consultar tickets y marcar uso únicamente dentro de su función operativa; como el dominio aún no modela asignación de personal a eventos, el MVP autoriza `STAFF` global para esos endpoints y lo documenta como **limitación que debe restringirse en producción**.
- El sistema puede responder 403 para objeto ajeno; cuando sea necesario evitar filtración sobre existencia de recursos, utilizar 404 de forma consistente según política definida y probada.

12. JWT: contrato y políticas

12.1 Claims mínimos

Claim	Contenido	Regla
`sub`	ID interno inmutable del usuario	Obligatorio y no vacío.
`iss`	`pulsepass-api` o valor configurado	Debe coincidir con issuer esperado.
`iat`	Fecha de emisión UTC	Rechazar fechas inaceptables según política.

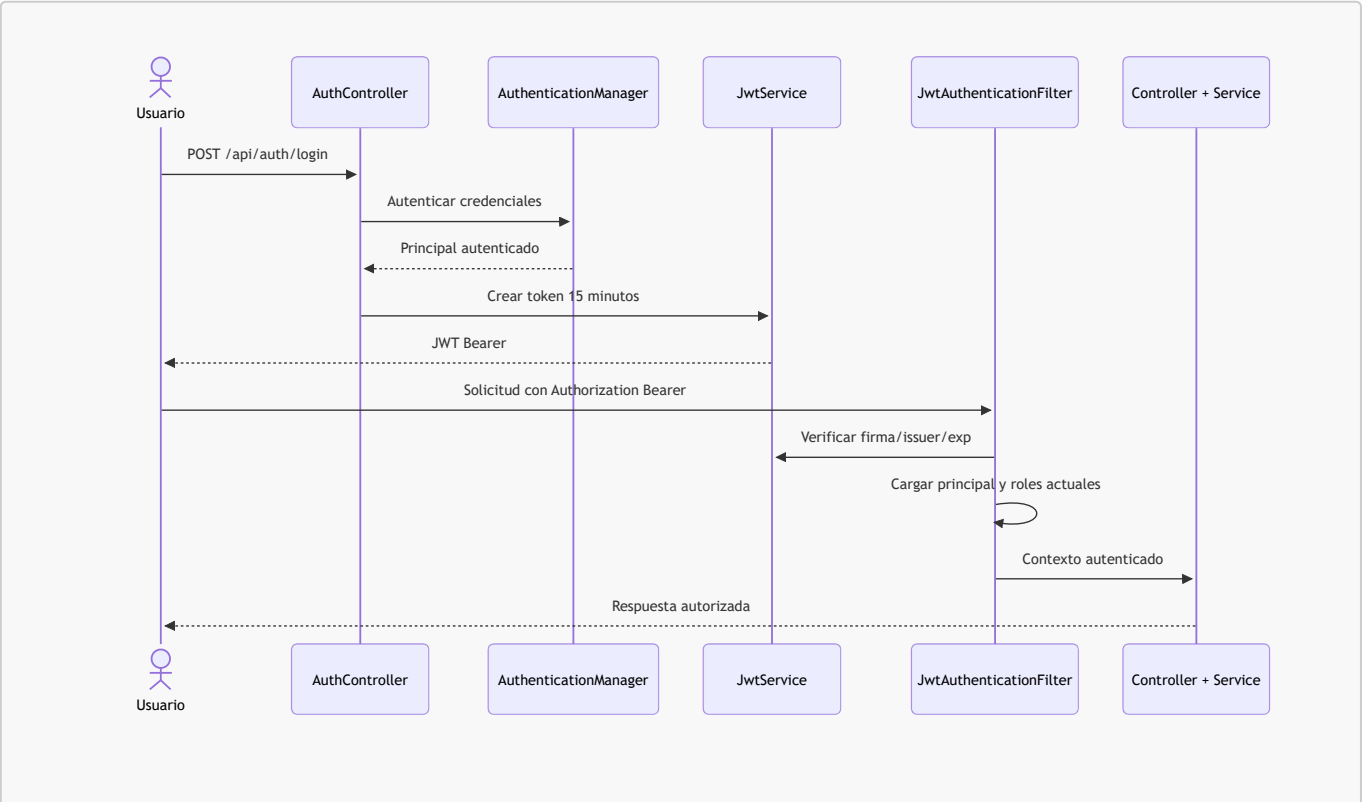
Claim	Contenido	Regla
`exp`	Fecha de vencimiento UTC	Obligatoria y futura.
`jti`	Identificador aleatorio del token	Obligatorio; no implica revocación instantánea.
`roles`	Opcional; lista de roles	Si aparece, **no sustituye** la carga del estado y roles actuales desde persistencia.

Preferencia: guardar únicamente identificador (`sub`) y metadatos; consultar roles actuales en `UserDetailsService`. De este modo, un token no conserva privilegios después de una revocación de rol si cada solicitud revalida el principal.

12.2 Firma

- Algoritmo: `HS256` explícito.
- Secretos: Base64 de **al menos 32 bytes aleatorios**; leer de la variable de entorno`PULSEPASS_JWT_SECRET_BASE64`; la aplicación debe fallar al arrancar si la clave está ausente o es insuficiente.
- Comparar algoritmo esperado y verificar criptográficamente firma antes de confiar en claims.
- No permitir `alg=none` ni token unsigned.
- JWT solo por HTTPS en redes reales; no escribirlo en logs.
- Para el MVP, una clave simétrica compartida por una sola aplicación es aceptable; para despliegues distribuidos considerar gestión de claves, identificadores de clave y rotación.

12.3 Ciclo de vida y revocación



La expiración limita la validez de tokens robados. Como no existe denylist, **logout del lado del cliente elimina su copia pero no invalida tokens ya emitidos** hasta vencer. El MVP reduce riesgo mediante vida corta y

consulta del estado/roles en cada request; revocación inmediata por `jti` se considera fuera de alcance.

13. Contratos de error y estado HTTP

Se **reutiliza**:

```
public record ErrorResponse(
    LocalDateTime timestamp,
    int status,
    String error,
    String message,
    Map<String, String> details
) {}
```

Caso	HTTP	`error` ejemplo	Responsable
Login incorrecto	401	`Unauthorized`	`AuthController` / autenticación.
Sin Bearer en ruta protegida	401	`Unauthorized`	`AuthenticationEntryPoint`.
JWT expirado o firmado incorrectamente	401	`Unauthorized`	Filtro + `AuthenticationEntryPoint`.
Usuario válido sin rol requerido	403	`Forbidden`	`AccessDeniedHandler`.
Usuario intenta ticket ajeno	403 (o 404, decisión consistente)	`Forbidden`	Servicio de autorización + handler.
JSON/Bean Validation inválido	400	`Bad Request`	Handler previo.
Usuario o evento no existente	404	`Not Found`	Handler previo.
Conflicto de reglas de compra/uso	409	`Conflict`	Handler previo.
Error inesperado	500	`Internal Server Error`	Handler previo; sin stack trace.

Ejemplo de 403:

```
{
  "timestamp": "2026-10-10T15:30:00",
  "status": 403,
  "error": "Forbidden",
```

```
  "message": "No tiene permisos para realizar esta operación",
  "details": {}
}
```

Importante: las excepciones antes de `DispatcherServlet` no quedan cubiertas automáticamente por `@RestControllerAdvice`; handlers de seguridad deben escribir la misma estructura JSON con `ObjectMapper` o delegar al `HandlerExceptionResolver` adecuado, cuidando que la respuesta se emita una sola vez.

14. Diseño técnico de referencia

Los siguientes fragmentos definen la intención arquitectónica. No sustituyen el trabajo de implementación ni los tests de integración.

14.1 Dependencias Maven

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-security</artifactId>
</dependency>
<dependency>
    <groupId>io.jsonwebtoken</groupId>
    <artifactId>jjwt-api</artifactId>
    <version>0.13.0</version>
</dependency>
<dependency>
    <groupId>io.jsonwebtoken</groupId>
    <artifactId>jjwt-impl</artifactId>
    <version>0.13.0</version>
    <scope>runtime</scope>
</dependency>
<dependency>
    <groupId>io.jsonwebtoken</groupId>
    <artifactId>jjwt-jackson</artifactId>
    <version>0.13.0</version>
    <scope>runtime</scope>
</dependency>
<dependency>
    <groupId>org.springframework.security</groupId>
    <artifactId>spring-security-test</artifactId>
    <scope>test</scope>
</dependency>
```

Conservar las dependencias previas (`webmvc`, validation, JPA, PostgreSQL, Flyway, MapStruct, JUnit, Mockito, Testcontainers) según el `pom.xml` existente. Verificar las coordenadas del starter web de Spring Boot 4 contra el proyecto base; no duplicarlo.

14.2 Configuración

```
pulsepass:
  security:
    jwt:
      issuer: ${PULSEPASS_JWT_ISSUER:pulsepass-api}
      secret-base64: ${PULSEPASS_JWT_SECRET_BASE64}
      access-token-ttl: PT15M
```

No subir un secreto real a `.env` versionado ni a `application.yml`. Documentar una plantilla `.env.example` sin secreto. En pruebas, usar clave exclusiva del perfil de tests.

14.3 Configuración simplificada de filtros

```
@Configuration
@EnableWebSecurity
@EnableMethodSecurity
public class SecurityConfig {

    @Bean
    SecurityFilterChain securityFilterChain(
            HttpSecurity http,
            JwtAuthenticationFilter jwtFilter,
            AuthenticationEntryPoint authEntryPoint,
            AccessDeniedHandler deniedHandler) throws Exception {

        http
            .csrf(csrf -> csrf.disable()) // Solo API Bearer sin cookies de
autenticación.
            .sessionManagement(sm ->
sm.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
            .formLogin(AbstractHttpConfigurer::disable)
            .httpBasic(AbstractHttpConfigurer::disable)
            .exceptionHandling(ex -> ex
                .authenticationEntryPoint(authEntryPoint)
                .accessDeniedHandler(deniedHandler))
            .authorizeHttpRequests(auth -> auth
                .requestMatchers(HttpMethod.POST, "/api/auth/login",
"/api/users").permitAll()
                .requestMatchers(HttpMethod.GET,
                    "/api/events/published", "/api/artists/**",
"/api/venues/**").permitAll()
                .requestMatchers(HttpMethod.POST, "/api/events").hasRole("ADMIN")
                .requestMatchers(HttpMethod.PATCH,
"/api/events/*/publish").hasRole("ADMIN")
                .requestMatchers(HttpMethod.POST,
"/api/events/*/artists/*").hasRole("ADMIN")
                .requestMatchers(HttpMethod.PATCH, "/api/tickets/*/use")
                    .hasAnyRole("STAFF", "ADMIN")
                .requestMatchers(HttpMethod.GET, "/api/events/*/tickets/paid")
                    .hasAnyRole("STAFF", "ADMIN")
                .anyRequest().authenticated())
```

```
            .addFilterBefore(jwtFilter,
UsernamePasswordAuthenticationFilter.class);

        return http.build();
    }
}
```

Advertencias obligatorias para la implementación:

1. Este fragmento no es la matriz completa: las comprobaciones de propiedad y la visibilidad de eventos no publicados deben implementarse en Services/seguridad a nivel de método. La ruta `GET /api/events/{eventCode}` no debe abrirse indiscriminadamente sin proteger borradores.
2. La ruta de eventos por artista también requiere filtrado de publicados si es pública.
3. La ruta `GET /api/venues/active` es pública y queda incluida en `/api/venues/**`; verificarla con una prueba HTTP explícita.
4. El orden de matchers importa: rutas específicas antes de patrones generales; `GET /api/events/published` antes de `{eventCode}`.
5. El filtro JWT debe dejar continuar requests sin header **solo** para que la cadena decida si la ruta es pública; si se suministra Bearer inválido, jamás autenticarlo por accidente.
6. Deshabilitar CSRF solo es válido mientras no se use autenticación enviada automáticamente por navegador, como cookies de sesión. Si cambia ese diseño, revisar CSRF.

14.4 Implementación JJWT — patrón obligatorio

```
SecretKey key = Keys.hmacShaKeyFor(
    Decoders.BASE64.decode(secretBase64)
);

String token = Jwts.builder()
    .subject(userId.toString())
    .issuer(expectedIssuer)
    .issuedAt(Date.from(now))
    .expiration(Date.from(now.plusSeconds(900)))
    .id(UUID.randomUUID().toString())
    .signWith(key, Jwts.SIG.HS256)
    .compact();

Claims claims = Jwts.parser()
    .verifyWith(key)
    .requireIssuer(expectedIssuer)
    .build()
    .parseSignedClaims(token)
    .getPayload();
```

Rechazar o manejar con respuesta 401 excepciones como `ExpiredJwtException`, `MalformedJwtException`, `SignatureException`, `UnsupportedJwtException`, así como errores de parseo y configuración. La validación debe imponer `exp`, `sub` y algoritmo esperado, no asumir su presencia por haber parseado la firma. Nunca usar APIs obsoletas de JJWT como `setSigningKey` / `parseClaimsJws` en código nuevo.

14.5 Estructura propuesta del proyecto

```
src/main/java/com/pulsepass/
├── controller/                    # existentes
│   └── AuthController.java        # NUEVO
├── service/                       # existentes
│   ├── AuthService.java           # NUEVO
│   └── AuthorizationService.java  # NUEVO
├── dto/
│   ├── request/LoginRequest.java
│   └── response/{AuthResponse,AuthenticatedUserResponse}.java
├── security/
│   ├── SecurityConfig.java
│   ├── JwtProperties.java
│   ├── JwtService.java
│   ├── JwtAuthenticationFilter.java
│   ├── CustomUserDetailsService.java
│   ├── SecurityPrincipal.java
│   ├── RestAuthenticationEntryPoint.java
│   └── RestAccessDeniedHandler.java
├── domain/                        # existentes + credenciales/roles
├── repository/                    # existentes + acceso mínimo a credenciales
├── exception/                     # se conserva GlobalExceptionHandler
└── mapper/                        # existentes

src/main/resources/db/migration/
└── V4__security_credentials_roles.sql

src/test/java/com/pulsepass/
├── security/
├── controller/
└── integration/
```

15. Casos de uso y escenarios verificables

UC-SEC-01 — Registro de Andrea

Dado que Andrea no tiene cuenta, **cuando** llama `POST /api/users` con datos válidos y contraseña de 12+ caracteres, **entonces** recibe 201 y el sistema persiste identidad, hash y `ROLE_USER` de forma atómica. No se expone el hash.

UC-SEC-02 — Login de Andrea

Dado que Andrea tiene una cuenta activa y contraseña correcta, **cuando** llama `POST /api/auth/login`, **entonces** recibe JWT válido con `sub` igual al ID persistente y caducidad de 15 minutos.

UC-SEC-03 — Consulta pública del evento

Dado el evento `CMF-2026` publicado, **cuando** un anónimo consulta `/api/events/published`, **entonces** recibe los eventos publicados sin token. Los borradores no son visibles públicamente.

UC-SEC-04 — Compra de ticket propia

Dado que Andrea está autenticada como `USER`, **cuando** envía `POST /api/tickets` con su correo, evento publicado `CMF-2026` y un tipo válido, **entonces** se mantienen todas las verificaciones del `TicketService` (edad/capacidad/estado), y la compra se permite solo para Andrea.

UC-SEC-05 — Intento de compra para Carlos

Dado que Andrea está autenticada, **cuando** envía en `PurchaseTicketRequest.userEmail` el correo de Carlos, **entonces** recibe 403 y **no** se crea ticket.

UC-SEC-06 — Consulta de ticket ajeno

Dado un ticket perteneciente a Carlos, **cuando** Andrea consulta su `ticketCode`, **entonces** recibe 403 (o 404 conforme a política única) y no obtiene información del ticket.

UC-SEC-07 — Uso por STAFF

Dado que Laura posee rol `STAFF` y un ticket válido aún no ha sido utilizado, **cuando** invoca `PATCH /api/tickets/{ticketCode}/use`, **entonces** se permite la acción y `TicketService.markAsUsed` valida la transición. Un `USER` recibe 403.

UC-SEC-08 — Publicación restringida

Dado que un `USER` intenta publicar el evento `CMF-2026`, **cuando** invoca `PATCH /api/events/CMF-2026/publish`, **entonces** recibe 403 y no se ejecuta el método de publicación.

UC-SEC-09 — JWT inválido

Dado un token vencido, manipulado o emitido por otro issuer, **cuando** llama una ruta protegida, **entonces** recibe 401 uniforme y no se invoca negocio.

UC-SEC-10 — Cuenta desactivada

Dado un token antes válido de un usuario desactivado, **cuando** intenta acceder, **entonces** el sistema vuelve a consultar el estado vigente y deniega el request.

16. Requisitos no funcionales

ID	Requisito	Criterio verificable
NFR-SEC-001	Confidencialidad de secretos	Sin password, hash, JWT ni clave HMAC en logs, respuestas o repositorio.
NFR-SEC-002	Disponibilidad ante tokens erróneos	Entrada Bearer malformada no produce excepción sin manejar ni HTTP 500.

ID	Requisito	Criterio verificable
NFR-SEC-003	Rendimiento	Evitar consultas N+1 para principal y roles; medir carga de autenticación.
NFR-SEC-004	Integridad	Contraseñas se escriben dentro de transacción de registro con usuario y rol.
NFR-SEC-005	Compatibilidad	Las 20 respuestas exitosas anteriores conservan DTO y código HTTP.
NFR-SEC-006	Trazabilidad	Registrar evento de login correcto/fallido sin credenciales, y denegaciones, con request ID.
NFR-SEC-007	Transporte	HTTPS obligatorio fuera de localhost y ambientes de laboratorio aislados.
NFR-SEC-008	Configurabilidad	Emisor, duración, clave y política CORS parametrizados.
NFR-SEC-009	Seguridad de privilegios	No existe registro público con roles `STAFF` o `ADMIN`.
NFR-SEC-010	Pruebas	Cobertura de matrices por método/rol/propiedad y regresión previa.

16.1 Controles operativos recomendados

Adoptar en producción límite de intentos de login, monitoreo de fallos, protección contra fuerza bruta, gestor de secretos, rotación HMAC y CORS de orígenes explícitos. El límite de intentos y rotación automatizada no forman parte del MVP académico; documentarlos como brechas evita confundir el taller con un sistema listo para producción.

17. Estrategia de pruebas

17.1 Unitarias (`JUnit 5`, `Mockito`, `AssertJ`)

- `JwtServiceTest`: firma correcta; claims correctos; expiración; issuer incorrecto; manipulación; token vacío; clave débil.
- `AuthServiceTest`: credenciales correctas, password incorrecto, usuario inactivo y emisión solo tras autenticación.
- `CustomUserDetailsServiceTest`: usuario existente, inexistente, roles actualizados, estado inactivo.
- `AuthorizationServiceTest`: dueño vs tercero; `STAFF` en uso; `ADMIN` en publicación; rechazo de correos ajenos.
- `JwtAuthenticationFilterTest`: sin Bearer, Bearer válido, Bearer inválido, doble header sospechoso, no sobrescribir autenticación establecida de forma insegura.

17.2 Pruebas de Controller/seguridad con `MockMvc`

Usar `@WebMvcTest` + `@Import` de configuración relevante o `@SpringBootTest` según el escenario, `spring-security-test`, `@WithMockUser`, `jwt` de pruebas cuando corresponda y `@MockitoBean` para dependencias.

No deshabilitar filtros para que los tests pasen: `@AutoConfigureMockMvc(addFilters = false)` invalidaría el propósito de probar seguridad.

Test ID	Dado / Acción	Esperado
T-SEC-001	Anónimo `GET /api/events/published`	200.
T-SEC-002	Anónimo `POST /api/events`	401.
T-SEC-003	`USER POST /api/events`	403, sin invocar Service.
T-SEC-004	`ADMIN POST /api/events` válido	201.
T-SEC-005	Anónimo `POST /api/auth/login` correcto	200 con Bearer.
T-SEC-006	Login con contraseña incorrecta	401 genérico.
T-SEC-007	Token expirado en ruta protegida	401 con `ErrorResponse`.
T-SEC-008	Token con firma alterada	401, sin invocar Service.
T-SEC-009	`USER` consulta ticket ajeno	403/404 según política acordada.
T-SEC-010	`USER` compra con email ajeno	403; 0 tickets creados.
T-SEC-011	`USER` cancela ticket propio	200 si reglas del Service lo permiten.
T-SEC-012	`USER` intenta usar ticket	403.
T-SEC-013	`STAFF` marca ticket válido como usado	200.
T-SEC-014	`ADMIN` publica evento	200.
T-SEC-015	`USER` busca datos de otro usuario	403/404.
T-SEC-016	Un token válido pertenece a usuario inactivo	401.
T-SEC-017	Anónimo consulta evento no publicado	No expone datos del borrador.
T-SEC-018	Request inválido pero autorizado	400 previo de Bean Validation.
T-SEC-019	Login o JWT da error	No devuelve stacktrace, password ni hash.
T-SEC-020	Pruebas REST previas con identidad apropiada	Mantienen códigos y DTOs exitosos.

17.3 Integración selectiva con PostgreSQL/Testcontainers

Validar migración Flyway, unicidad de roles, persistencia de hash BCrypt, login contra usuario real, transacción atómica de registro y consulta segura de propietario. En tests usar contraseñas y secretos exclusivamente ficticios y no reciclar credenciales de producción.

17.4 Cobertura obligatoria

Para cada una de las 20 operaciones, documentar al menos: rol permitido, rol prohibido, anónimo y, cuando exista ownership, mismo propietario / tercero. Los tests previos de negocio **siguen vigentes** y deben adaptarse para configurar identidad en lugar de eliminarse.

18. Trazabilidad requisito → componente → pruebas

Requisito	Componente	Pruebas asociadas
FR-SEC-AUTH-001	AuthService, UserService, Flyway	T-SEC-005 + integración registro
FR-SEC-AUTH-002/003	AuthController, AuthenticationManager	T-SEC-005/006
FR-SEC-AUTH-004/009	CustomUserDetailsService, filtro	T-SEC-016
FR-SEC-AUTH-005/006/007	JwtService, JwtAuthenticationFilter	T-SEC-007/008
FR-SEC-VEN-01/02	SecurityConfig	Matriz GET públicos
FR-SEC-EVT-01..06	SecurityConfig, AuthorizationService, EventService	T-SEC-001/002/003/004/014/017
FR-SEC-ART-01..03	SecurityConfig	Matriz GET públicos
FR-SEC-USR-01..03	AuthService, UserService, AuthorizationService	T-SEC-005/015
FR-SEC-TKT-01..06	AuthorizationService, TicketService	T-SEC-009..013
SEC-009 / NFR-SEC-002	EntryPoint, AccessDeniedHandler	T-SEC-007/008/019
NFR-SEC-005/010	Suites previas + MockMvc	T-SEC-020

19. Plan de implementación incremental (taller)

El orden recomendado evita introducir varias fuentes de fallos al mismo tiempo:

Etapa	Actividad	Evidencia de entrega
1. Contrato	Mapear las 20 rutas existentes y confirmar políticas de exposición.	Matriz de autorización revisada.
2. Persistencia	Flyway V4+, credenciales/roles y estrategia para legados.	Migraciones + tests Testcontainers.
3. Passwords	BCrypt y `CustomUserDetailsService`.	Password hash y pruebas de autenticación.
4. Auth	Login con `AuthenticationManager`, DTOs y `AuthService`.	`POST /api/auth/login` probado.
5. JJWT	`JwtService` con firma, parseo y expiración.	Test de claims y rechazo de tokens.
6. Seguridad HTTP	Filter chain, Bearer, roles, 401 y 403.	Smoke tests de las 20 rutas.

Etapa	Actividad	Evidencia de entrega
7. Ownership	Asegurar búsqueda, compra y cancelación de tickets.	Pruebas contra IDOR.
8. Integración	Ejecutar suites previas y escenarios completos.	Reporte de pruebas y DoD.

20. Riesgos, mitigaciones y decisiones pendientes

Riesgo	Impacto	Mitigación
Registro original carece de contraseña	Login imposible sin evolución de contrato.	Agregar `password` y migración de credenciales explícita.
Usuarios legados sin contraseña	Usuarios preexistentes no pueden iniciar sesión.	Activación segura fuera de flujo de registro; sin contraseñas por defecto.
Confiar en `email` de compra	Ticket a nombre de tercero.	Contrastar principal inmutable y propietario en Service.
JWT con roles obsoletos	Acceso luego de retirar privilegios.	Revalidar roles/estado contra base de datos.
`permitAll()` de un GET de eventos	Filtración de borradores.	Filtrado por publicación y test específico.
Token robado sigue válido	Uso hasta expiración.	15 minutos, HTTPS y futura revocación centralizada.
Staff sin vínculo al evento	Acceso global a operaciones de tickets.	Declarar brecha del MVP y diseñar `EventStaffAssignment` en evolución futura.
Exclusiones por matcher mal ordenado	Ruta privilegiada pública accidentalmente.	Pruebas de matriz y revisión de patrones.
Un handler no captura errores del filtro	HTTP incorrecto o 500.	EntryPoint y AccessDeniedHandler con `ErrorResponse`.

20.1 Decisiones abiertas, sin bloquear el MVP

- Verificar que las pruebas de regresión incluyan `GET /api/venues/active` y `GET /api/venues/{code}`.
- Decidir si un ticket ajeno siempre se reportará como 403 o se ocultará con 404.
- Acordar procedimiento de alta controlada de `STAFF`/`ADMIN` en tests y entornos no productivos.
- Determinar cómo implementar acceso de `STAFF` limitado a un evento cuando el dominio se amplíe.
- Documentar si las búsquedas de eventos por código y artista requieren nuevos métodos filtrados por publicación o la extensión segura de consultas existentes.

21. Definition of Ready (DoR)

- [ ] Compilan las fases previas y existen sus 20 endpoints.
- [ ] Se conocen los nombres exactos de `User`, su ID, estado, repositorio y DTOs.
- [ ] Se han verificado las dos rutas de `VenueController` (`/{code}` y `/active`).
- [ ] Existe una migración Flyway siguiente disponible.
- [ ] Se aprueba matriz pública / `USER` / `STAFF` / `ADMIN`.
- [ ] El equipo entiende autenticación, autorización, JWT y ownership.
- [ ] Se definió secret de desarrollo **no compartido** y mecanismo para tests.

22. Definition of Done (DoD)

- [ ] Se mantienen las 20 rutas heredadas y sus respuestas exitosas.
- [ ] Registro crea usuario + contraseña hasheada + rol `USER` de forma atómica.
- [ ] Login correcto emite JWT válido y login erróneo devuelve 401.
- [ ] `JwtService` valida firma, issuer, `sub`, `exp` y algoritmo.
- [ ] API stateless; Bearer obligatorio salvo excepciones públicas explícitas.
- [ ] `USER`, `STAFF` y `ADMIN` tienen reglas probadas.
- [ ] Ningún `USER` consulta, compra para o cancela tickets de terceros.
- [ ] Ni eventos borrador ni datos privados quedan expuestos por GET público.
- [ ] `ErrorResponse` es consistente para 400, 401, 403, 404, 409 y 500.
- [ ] No existe secreto hardcodeado, contraseña sin hash ni exposición de datos sensibles.
- [ ] Flyway se ejecuta con éxito sobre esquema nuevo y esquema ya migrado.
- [ ] Tests unitarios, MockMvc con filtros e integración pasan.
- [ ] No se modifican reglas de negocio existentes sin requerimiento explícito.
- [ ] README incluye variables, roles de prueba, endpoints y comandos de verificación.

23. Escenario final de demostración

Contexto heredado: venue `VEN-SMR-01`, festival `CMF-2026`, artista Solar Beat, usuarios Andrea y Carlos, y personal de operación Laura/Miguel conforme a los escenarios previos. Los roles en esta fase se asignan solo a las cuentas de demostración, sin alterar el dominio de eventos.

1. Andrea se registra con contraseña; verifica respuesta sin hash.
2. Andrea inicia sesión y recibe JWT con `expiresIn = 900`.
3. Un anónimo consulta los eventos publicados y encuentra `CMF-2026`.

4. Andrea compra su propia entrada cuando se cumplen las reglas comerciales.
5. Andrea intenta consultar o cancelar una entrada de Carlos: acceso denegado.
6. Andrea intenta publicar `CMF-2026`: recibe 403.
7. Un administrador publica el evento y asocia Solar Beat mediante rutas existentes.
8. Laura (`STAFF`) marca una entrada como utilizada; una segunda utilización es rechazada por la regla de negocio heredada.
9. Se altera un carácter de un JWT y se comprueba 401.
10. Se desactiva un usuario con token aún vigente y se comprueba que el próximo request no es autorizado.

Resultado esperado: el mismo flujo funcional de PulsePass de fases 1–3 permanece operativo, pero todos los accesos sensibles están protegidos y son verificables.

24. Fuentes técnicas y referencias

- Spring Security — Authorization of HTTP requests
- Spring Security — Architecture
- Spring Security — CSRF
- JJWT — repositorio oficial, instalación y API
- OWASP — Authentication Cheat Sheet
- OWASP — Password Storage Cheat Sheet
- OWASP — JSON Web Token Cheat Sheet for Java
